Chapter 12

Recoding

by Lorraine Gaudio

Version May 29, 2026

Contents

1 Overview

In this chapter, you will learn how to use **vectorized conditional functions** to return a result for every element in a vector. You will compare `ifelse()` and `case_when()`, learn when each is most useful, and see how conditional rules can be used inside `mutate()` to create or overwrite columns in a data frame. You will also continue practicing in Quarto, the course's reproducible workflow ( ) for planning and writing code, checking output, and written interpretation in one document. You move from simply selecting data

to actively transforming it into more useful variables. `ifelse()` returns values based on a logical test, `case_when()` applies rules in order using the first match, and `mutate()` creates new columns from existing variables or modifies existing ones.

This chapter matters because data analysts rarely work only with raw variables. In real analysis, you often need to recode categories, group values into meaningful labels, convert units, and create cleaner columns from existing data before you can summarize, visualize, or model it. Learning to do this with readable R code will help you build data sets that are easier to interpret, easier to check, and easier for other people to follow. `dplyr` describes these kinds of recoding and column-building tasks as common data manipulation work, especially when creating entirely new columns from existing ones

By the end of Chapter 12 you will be able to:

- Distinguish between logical subsetting, subassignment, and vectorized conditionals.
- Use `ifelse()` to make simple two-way decisions across a vector.
- Use `case_when()` to apply multiple conditional rules in a clear order.
- Explain why the order of rules matters in `case_when()`.
- Recode numeric values into meaningful label".
- Use `mutate()`to create new columns from existing columns.
- Overwrite an existing column by reusing its name inside `mutate()`.
- Use `mutate()` with arithmetic expressions to create transformed variables such as unit conversions.
- Continue using Quarto to organize code, output, and written interpretation in a reproducible analysis document.

Keep these goals in mind as you move through each section.

2 Quarto: Part 2

In Chapter 11, you learned how to create a Quarto file, keep the YAML header, use Markdown headings, write memos outside code chunks, and control chunks with #| options. In this chapter, you will build two more Quarto skills that make your notes more readable and more reproducible:

1. add a table of contents and section numbering
2. use inline R code inside your sentences

Create a Quarto File ()

1. For those of you who **have GitHub** and a course project, navigate to your course folder and open your course project. This will open RStudio. Click "Pull" in the Git pane to get the latest version of the course materials. After completing work, save, stage, commit, and push.

If you **don't have GitHub** or a course project, just open RStudio and set your working directory to your course folder.

2. Open a new Quarto document in your course folder and save it as: `chapter12_notes.qmd` (`File > New File > Quarto Document`).

2.1 toc: true ()

As your notes get longer, it becomes harder to scroll and find what you need. Quarto can automatically build a table of contents from your headings, and it can also number your sections for you. The main YAML options for this are `toc: true`, `toc-depth:`, and `number-sections: true`. These options are commonly placed in the document options near the top of the YAML.

Use this updated YAML header for Chapter 12:

```yaml
---
title: "Chapter 12 Notes"
subtitle: "mutate() and case_when()"
author: "YOUR NAME"
date: today
toc: true
toc-depth: 2
number-sections: true
format:
  html:
    code-fold: true
execute:
  warning: true
  message: true
---
```

2.1.1 What each part does:

- `toc: true` adds a table of contents automatically.
- `toc-depth: 2` tells Quarto to include level 1 and level 2 headings in that table of contents.
- `number-sections: true` adds section numbers to your headings in the rendered document.

The `toc-depth: 2` is a good default. It keeps your notes organized without making the table of contents too crowded.

2.2 Headings ()

You will want to use headings to see how the table of contents work. Create a simple outline for Chapter 12 using headings. Try using the following structure:

```
# Quarto Notes

# Load Packages

# Vectorized Conditionals

## ifelse()

## case_when()

## More than Two Categories

### Nested ifelse()

### Multiple Rule case_when()

# mutate()

## Column Copy

### mutate() with Arithmetic

### case_when() inside mutate()
```

```
## Overwrite Column Values
```

Render to HTML often to see the results. *Render is like knit or preview.*

2.3 Dynamic memoing ()

So far, you have used code chunks to run R code. Quarto also lets you place small pieces of R code **directly inside normal sentences**. This is called inline code. Inline code is useful when you want your written memo to automatically include a value computed by your R code. Quarto evaluates inline code during rendering, so the sentence stays up to date with your analysis.

In Quarto, inline code uses one pair of backticks and the engine name inside the backticks. For R, the pattern is:

```
`{r} expression`
```

Using inline R code helps ensure that your report always shows the exact values used to in calculations.

2.3.1 Inline Example

Imagine you want to write a sentence that says how many rows are in the filtered `mtcars` dataset. Instead of hardcoding the number, you can use inline code to compute it.

```
#  Type this in a Quarto code chunk
library(dplyr)
num_rows <- mtcars %>%
  filter(cyl == 6) %>%
  nrow()
```

Then, in your written memo, you can write the expression:

```
The filtered dataset has `{r} num_rows` rows.
```

When rendered to HTML, this will show as:

The filtered dataset has 7 rows.

In this chapter, use inline R code when you want your memo to report a computed value such as a vector length, category count, or converted measurement.

For this chapter, practice setting up your notes file so that:

1. the YAML includes a table of contents and section numbering
2. your headings organize the chapter clearly
3. your code chunks do the main work
4. your memos use inline R code whenever you want to report a value from your analysis

Now that you have created (*and saved*) your quarto file, follow along this guided practice and taking notes. Fill in each section under the headings.

Tip: Test Render your document often to see how your notes are coming together.

3 Packages

In this chapter, you will use `dplyr` verbs and `dslabs` datasets.

Remember that you only need to install once `install.packages("dplyr")`; `install.packages("dslabs")`.

Load the required packages at the beginning of your QMD.

```
library(dslabs)   # Data science labs package
library(dplyr)    # Data manipulation package
```

4 Vectorized Conditional Functions

In previous chapters, you used logical conditions to keep some values and drop others, or to replace selected values in place. In this chapter, you will use logical conditions in a third way: to return a result for every element. That is what functions like `ifelse()` and `case_when()` do.

Logical conditions in R can do more than *select* values. They can also be used to return a new vector by assigning an output to each element based on one or more rules. Functions such as `ifelse()` and `case_when()` are called **vectorized conditional** functions because they evaluate conditions element-by-element across a vector and return a new vector of the same length.

This section builds on ideas you already know. In earlier chapters, you used conditions to keep values or rows with logical subsetting, and to replace selected values with subassignment. Now you will use conditions in a new way: not to drop or replace selected cases, but to return a result for every position. Those results are often used inside `mutate()` to create or modify columns.

Subsetting keeps some values and drops others. Vectorized conditionals, by contrast, keep all positions and assign each one a result. In other words, subsetting answers "What do I keep?" while vectorized conditionals answer "What value should I return for each element?"

4.1 Review Subsetting with Conditions

Subsetting answers the question, "Which values *or rows* do I keep?" With a vector, **logical indexing** uses a pattern of `TRUE` and `FALSE` values to keep or modify selected positions.

```
#  Review
## Create vector x
x <- 1:6
## logical vector
x > 3
```

```
## [1] FALSE FALSE FALSE  TRUE  TRUE  TRUE
```

```
#  Review
## logical subsetting
x[x > 3]
```

```
## [1] 4 5 6
```

`x[x > 3]` subsets `x` to keep only values greater than `3`.

Notice: You used the same idea with data frames in Chapter 11. The `filter()` function also applies a logical condition, but at the row level, keeping rows that satisfy the condition and dropping the rest.

4.2 Review Subassignment

Subsetting is used to *replace* values, called **subassignment**. When you assign a value to a subset of an object, you are replacing the values in those positions.

```
#  Review
## replaces values
x[x > 3] <- 99
```

```
##   print result
x
```

```
## [1]  1  2  3 99 99 99
```

`x[x > 3]` subsets `x` to keep only the values greater than `3`, and `x[x > 3] <- 99` uses logical indexing to replace those matching values with `99`.

This review example uses a condition to identify matching cases. The difference is what happens next:

- Subsetting keeps only matching values or rows.
- Subassignment changes only matching values.
- Vectorized conditionals return a result for every position.

That is the key new idea in this section. Vectorized conditional functions such as `ifelse()` and `case_when()` still test conditions element-by-element, but they do not drop nonmatching cases. Instead, they evaluate a condition for each element and return a *new vector* with one result for every position.

When this new vector is used inside `mutate()`, a function shown at the end of this chapter, it becomes a new column rather than a subset of the old one.

4.3 Simple `ifelse()`

Base R describes `ifelse()` as returning a value with the same shape as the test, filled from `yes` or `no`.

The SYNTAX: `ifelse(test, yes, no)`

- **test** – logical vector (`TRUE`/`FALSE`/`NA`)
- **yes** – value to insert where test is `TRUE`
- **no** – value to insert where test is `FALSE`.
- **NA** – if `NA`, the result is `NA` at that position.

Label the vector of numbers greater than 3 as “Large” and others as “Small”.

```
#  Type this in a Quarto code chunk:
x <- c(3, 2, 3, 6, 5)
```

```
#   Pre-check
length(x)
```

```
## [1] 5
```

Use this inline code to dynamically report the length of `x` within your memo:

```
 `x` is a vector with the length of `{r} length(x)` rows.
```

When rendered to HTML, this should read:

x is a vector with the length of 5

```
#  Type this in a Quarto code chunk:
ifelse(test = x > 3, yes = "Large", no = "Small")
```

```
## [1] "Small" "Small" "Small" "Large" "Large"
```

The output positions for 3 and under are "Small" and numbers higher than 3 are "Large".

```
#   Verify
length(ifelse(x > 3, "Large", "Small"))
```

```
## [1] 5
```

The result has the same length as x. Nothing was dropped. Each position got a label.

Note: Arguments, such as ***test****,* ***yes****, and* ***no*** *in* ***ifelse()****, are often not required to be named inside of functions.*

Tip: Read this `ifelse()` statement like 'If the vector is greater than 3, then return "Large", otherwise return "Small"."

- `x[x > 3]` logical subsetting keeps only matches
- `x[x > 3] <- 99` subassignment replaces only matches
- `ifelse(x > 3, "Large", "Small")` returns a new vector with a value for every position based on the condition.

4.4 Simple `case_when()`

`dplyr` describes `case_when()` as a general vectorized if-else that evaluates rules in order and uses the first match for each element. This makes it especially useful when you have several categories to assign.

`dplyr::case_when()` can do exactly what `ifelse()` is doing.

The SYNTAX: `case_when(condition1 ~ value1, condition2 ~ value2, .default = default_value)`

The `~` symbol means "maps to" or "yields".

The `.default` argument specifies the value to return when no conditions are `TRUE`. If you omit `.default` and no conditions match, `case_when()` will return `NA` for those positions.

Label the vector of numbers greater than 3 as "Large" and others as "Small".

```
#  Type this in a Quarto code chunk:
case_when(c(3, 2, 3, 6, 5) > 3 ~ "Large", .default = "Small")
```

```
## [1] "Small" "Small" "Small" "Large" "Large"
```

The output positions for 3 and under are "Small" and numbers higher than 3 are "Large".

- `case_when(c(3, 2, 3, 6, 5) > 3 ~ "Large", .default = "Small")` returns a new vector with a value for every position based on the condition.

Why do we need both `ifelse()` and `case_when()`? `case_when()` is usually clearer once you have 3+ categories.

4.5 More Than Two Categories

`ifelse()` is designed for two categories. If you have more than two, you can nest `ifelse()` statements inside each other, but this quickly becomes hard to read. `case_when()` is designed for multiple categories and is much easier to read when you have more than two conditions.

```
#   Setup
#  Type this in a Quarto code chunk:
scores <- c(55, 65, 75, 85, 95)
```

Re-code scores less than 60 as "Low", scores less than 80 as "Medium", and all others as "High".

4.5.1 Nested `ifelse()`

Re-code scores less than 60 as "Low", scores less than 80 as "Medium", and all others as "High" using nested `ifelse()`.

```
#  Type this in a Quarto code chunk:
ifelse(scores < 60, "Low",
       ifelse(scores < 80, "Medium", "High"))
```

```
## [1] "Low"    "Medium" "Medium" "High"   "High"
```

This works, but the logic is nested inside the no argument of the first `ifelse()`, which becomes harder to read as the number of categories grows.

4.5.2 Multiple Rule `case_when()`

`case_when()` avoids the ugly nesting that happens with repeated `ifelse()` calls. It is a general vectorized if-else: R checks the rules from top to bottom and uses the first rule that matches each element. That is why the order of the rules matters.

Code scores less than `60` as `"Low"`, scores less than `80` as `"Medium"`, and all others as `"High"` using `case_when()`.

```
#  Type this in a Quarto code chunk:
case_when(scores < 60 ~ "Low",
          scores < 80 ~ "Medium",
          TRUE        ~ "High")
```

```
## [1] "Low"    "Medium" "Medium" "High"   "High"
```

This reads more like a set of classification rules: first check for `"Low"`, then check for `"Medium"`, and assign `"High"` to everything else.

The order of the rules matters because `case_when()` uses the first condition that matches.

```
#  Type this in a Quarto code chunk:
case_when(
  scores < 80 ~ "Medium",
  scores < 60 ~ "Low",
  TRUE       ~ "High"
)
```

```
## [1] "Medium" "Medium" "Medium" "High"   "High"
```

This does **not** work as intended. Any score less than `60` also satisfies `score < 80`, so those values are labeled `"Medium"` before R ever checks the `"Low"` rule.

Tip: write rules from most specific to more general, and end with a default rule such as `TRUE ~ "High"`.

---

5 `mutate()`

So far, `case_when()` has returned a new vector of labels. In real data analysis, you usually want to store that result in a data frame as a new column. That is what `mutate()` does.

`mutate()` creates new columns that are functions of existing columns. It can also modify an existing column if you reuse the same column name.

The SYNTAX:

`mutate(data, new_col = expression)`

Or with piping

`name_data <- data %>% mutate(new_col = expression)`

- `data` is the data frame you are working with.
- `new_col` is the name of the new column you want to create.
- `expression` is the vector or calculation you want to store in that column.

You can think of `mutate()` as saying: "Take this data frame, and add a new column based on these existing columns."

The key idea is this:

- A new name on the left side of `=` adds a new column.
- An existing name on the left side of `=` overwrites that column.
- The right side of `=` tells R what values to put there.

5.1 Column Copy

Create a new column that is a copy of `am` in `mtcars`.

```
#   Pre-Check
names(mtcars) # Check the column names
```

```
##  [1] "mpg"  "cyl"  "disp" "hp"   "drat" "wt"   "qsec" "vs"   "am"   "gear"
## [11] "carb"
```

```
#  Type this in a Quarto code chunk:
mtcars2 <- mtcars %>%
  mutate(new_column_copy = am)
```

In a new data set called `mtcars2`, a **new** column named `new_column_copy` is created that is identical to the `am` column.

```
#  Verify
names(mtcars2) # Check the column names again
```

```
##  [1] "mpg"             "cyl"             "disp"            "hp"
##  [5] "drat"            "wt"              "qsec"            "vs"
##  [9] "am"              "gear"            "carb"            "new_column_copy"
```

This simple example shows that `mutate()` can create a new column from an existing one. The right side of `mutate()` is for calculations or rule-based expressions on existing columns. We'll start by using calculations inside `mutate()` to create a new column.

5.1.1 `mutate()` with Arithmetic

Doing unit conversions is a common data analyst task. Here it is in action.

```
#   Setup
measurements <- data.frame(height_in = c(60, 65, 70))
measurements
```

```
##   height_in
## 1        60
## 2        65
## 3        70
```

Use this inline code to dynamically report within your memo:

```
The `measurements` data frame has one column called `{r} colnames(measurements)`.
```

When rendered to HTML, this should read:

The measurements data frame has one column called height_in

Create a new column called height_cm that converts inches to centimeters (1 inch = 2.54 cm).

```
#  Type this in a Quarto code chunk:
measurements %>%
  mutate(height_cm = height_in * 2.54)
```

```
##   height_in height_cm
## 1        60     152.4
## 2        65     165.1
## 3        70     177.8
```

mutate(height_cm = height_in * 2.54) creates a new column called height_cm. Each value in height_cm is calculated from the matching value in height_in.

5.1.2 case_when() inside mutate()

The right side of mutate() can also be a rule-based expression. One common example is case_when() which lets you build a new categorical variable from an existing numeric variable.

```
#   Setup
student_scores <- data.frame(scores = c(55, 68, 72, 83, 91))
```

Create a new column called score_level that labels each score from scores as "Low", "Medium", or "High".

```
#  Type this in a Quarto code chunk:
student_scores <- student_scores %>%
  mutate(
    score_level = case_when(
      scores < 60 ~ "Low",
      scores < 80 ~ "Medium",
      TRUE        ~ "High"
    )
  )
```

mutate() adds a new column called score_level. The values in that column come from the rules inside case_when().

- student_scores is the data frame.
- mutate(...) adds a new column.
- score_level = names that new column.
- case_when(...) decides what label to return for each row based on the value in scores.

```
#  Verify
head(student_scores)
```

```
##   scores score_level
## 1     55         Low
## 2     68      Medium
```

```
## 3     72      Medium
## 4     83        High
## 5     91        High
```

Use this inline code to dynamically report within your memo:

```
 `student_scores` is a dataframe with columns names: `{r} names(student_scores)`.
```

When rendered to HTML, this should read:

`student_scores` is a dataframe with columns names: scores, score_level

Or, you can get fancy with inline code and write and "and" between the column names:

```
 `student_scores` has the columns `{r} paste(names(student_scores_labeled), collapse = " and ")`.
```

When rendered to HTML, this should read:

`student_scores` has the columns scores and score_level.

The `mutate()` + `case_when()` duo is a powerful way to create new variables based on complex rules. You can have as many conditions as you need, and you can use any existing columns in your data frame to build those rules

In some situations, you may want to overwrite an existing column instead of creating a new one. That is also possible with `mutate()`, as the next section shows.

5.2 Overwrite Column Values

`mutate()` will overwrite an existing column by reusing the same column name on the left as the right of `=`. We'll use `case_when()` to recode the values in the `am` column of `mtcars` from numeric to categorical labels.

Recode the `am` column in `mtcars` from 0/1 to "Automatic"/"Manual".

```
#   Pre-check
unique(mtcars$am)
```

```
## [1] 1 0
```

`am` is a numeric column with two unique values: `0` and `1`.

```
#  Type this in a Quarto code chunk:
cars_overwrite <- mtcars %>%
  mutate(
    am = case_when(
      am == 0 ~ "Automatic",
      TRUE ~ "Manual")
    )
```

```
#  Verify:
unique(cars_overwrite$am)
```

```
## [1] "Manual"    "Automatic"
```

Use this inline code to dynamically report within your memo:

```
 The `am` column in `cars_overwrite` has the unique values: `{r} unique(cars_overwrite$am)`.
```

When rendered to HTML, this should read:

The `am` column in `cars_overwrite` has the unique values: Manual, Automatic.

The `am` column has been *changed* from `0/1` to `"Automatic"/"Manual"`.

Overwrite Key Takeaway

In every example above, `mutate()` followed the same pattern:

- the left side of `=` named the output column
- the right side of `=` created its values

When the left side is a new name, R adds a new column. When the left side reused an existing name, R overwrites that column.

6 Summary

In this chapter, you learned that logical conditions in R can do more than keep or replace values: they can also return a new result for every element. You practiced this with `ifelse()` for simple two-way decisions and `case_when()` for multi-rule classification, then used `mutate()` to store those results as new or overwritten columns in a data frame. You also continued building your Quarto workflow by adding a table of contents, section numbering, and inline R code so your notes can report values dynamically and stay reproducible.

7 Chapter Terms

Arguments: Inputs that you provide to functions to customize their behavior. Arguments are specified within the parentheses of a function call and can modify how the function operates or what data it processes.

Extraction: Retrieving a single component from an object—such as one list element or one data-frame column—so you get the "thing itself" (often a simplified result like a vector). Extraction is commonly done with `[[ ]]` (extract one element by position or name) or `$` (extract one column by name).

Logical Indexing: Indexing a vector using a logical vector of `TRUE`/`FALSE` values (for example, `x[x > 0]`) so R keeps the elements where the index is `TRUE` and drops those where it is `FALSE`. If the logical index is a different length, R may recycle it, which can lead to surprising results.

Logical Vector: A vector that stores Boolean elements: `TRUE` or `FALSE` (and sometimes `NA` for missing). Logical vectors are commonly produced by comparisons (for example, `x > 0`) and are heavily used for filtering and conditional code.

Overwrite: A common type of subassignment where you change existing values (you replace what is already there).

Subsetting: Selecting a smaller piece of an R object using indices (positions, names, or a logical mask). With data frames, subsetting most commonly uses `[` in the form `df[rows, cols]` and typically preserves the object's structure (e.g., you still get a data frame when you subset rows and columns).

Vectorized conditional: A function or expression that evaluates a condition element-by-element across a vector and returns an output for every element, producing a new vector of the same length. Unlike subsetting, which keeps some values and drops others, vectorized conditionals such as `ifelse()` and `case_when()` keep all positions and assign each one a result based on the condition.

8 Practice Space

This Practice Space helps you turn Chapter 12 concepts into a reproducible analysis workflow. In this chapter, you practiced using vectorized conditionals to return a result for every position, using `case_when()` in the correct order, and using `mutate()` to create new columns or overwrite existing ones. You also continued building your Quarto workflow by using headings, memoing, inline R code, and frequent rendering to HTML.

This work matters in this course because you are not only learning to write R code. You are learning to document your reasoning, check your results, and produce work that can be re-run and understood later. That connects directly to learning outcomes skills: reproducible workflow, problem-solving with code, and memo-based learning with brief AI disclosure when used.

These tasks give you practice with the main skills from Chapter 12. These are common data-analysis tasks. Analysts often need to recode values into clearer labels, convert units, and build cleaner variables before they can summarize, graph, or model data.

8.1 Setup

1. Create a new Quarto file in your course folder and save it as:

- `chapter12_practice.qmd`
- Test Render your document often to see how your Practice Space report is coming together.
- Submit `chapter12_practice.html`

2. Use the heading structure taught earlier in the chapter. Your Quarto setup for Chapter 12 should include headings, section numbering, and a table of contents, and you should render often to check both formatting and errors.

3. Library it up!

- Make sure there is script in your document that loads `dplyr` and `dslabs` packages so their functions / datasets load.

4. Memo your work with the // /

- a memo before the code chunk that states the goal of the code and what you are trying to learn or understand
- the code chunk itself
- any needed checks after the code
- a memo after the code chunk that explains what you learned, discovered, or confirmed
- answers to all comment questions included in the task

If you used help while working, document it in a memo. For example, if you used an LLM or another outside resource to understand an error or concept, briefly disclose that use and explain what help you received. Type all code yourself. This is part of the course expectation for AI-supported learning and memo-based workflow.

Follow this template for each coding task.

What is the goal of the code? What am I trying to learn, check, or demonstrate?

```
#Code
your_code <- "here"
```

```r
#  Include Checks after and pre-checks as needed
colnames(your_code)
head(your_code)
table(your_code)
class(your_code)
unique(your_code)
```

/ What I learned / Next step. 1–2 sentences.

8.2 Task 1

Concept Check

Which method returns a new vector with one result for every position?

a. Vectorized conditional functions such as `ifelse()` and `case_when()`

b. Logical subsetting using `[ ]`

c. Subassignment using `[ ] <-`

d. `filter()`

8.3 Task 2

Simple `ifelse()`

Use `ifelse()` to label each `height` in the dataset `heights` as `"Tall"` if `height` greater than `68` and `"Not Tall"` otherwise. Store the result as `height_label`.

```r
#  pre-check
?dslabs::heights
heights <- dslabs::heights
names(heights)
```

Which column contains height values in the dataframe `heights`? Should "Tall" be assigned to the yes or no argument in the ifelse statement? Why?

Restate the Goal. What are you trying to learn, check, or demonstrate?

```r
#  Code
height_label <- _____(
  test = _____$_____ > 68,
  yes = ______,
  no = ______)
```

```r
#  Check
table(height_label)  # expect "Tall" and "Not Tall"
```

Comment on the output. Do you see the expected labels? How many "Tall" and "Not Tall" are there? / What did you learn or notice from the output? Did the code work as expected? If you needed help, what kind of help did you use?

8.4 Task 3

Recode with `mutate()` and `case_when()`

Create `mpg_class` with three labels using `case_when()`: "High MPG" if `mpg` greater than `25`; "Low MPG" if `mpg` less than `15`; otherwise "Medium MPG".

Restate the Goal. What are you trying to learn, check, or demonstrate?

```
#  Code
mpg_recode <- mtcars %>%
  ______(mpg_class = _____(
    mpg > 25 ~ "_______",
    mpg < 15 ~ "_______",
    TRUE     ~ "_______"
  ))
```

```
#  Checks
table(mpg_recode$____)  # expect 3 levels
```

Comment on the output. Do you see the expected labels? How many "High MPG", "Medium MPG", and "Low MPG" are there? What did you learn or notice from the output? Did the code work as expected? If you needed help, what kind of help did you use?

8.5 Task 4

Recode: Name Focus

With the dataset `stars` from the `dslabs` package, classify each `star` by temperature range in a new column called `temp_group`. Store the result as `stars_recode`.

Use these rules: Temperatures

- less than 4,000 are "Cool",
- between 4,000 and 7,500 are "Warm", and
- all stars more than 7,500 are "Hot".

```
#  pre-check
?dslabs::stars
stars <- dslabs::stars

# View(stars)
colnames(stars)
summary(stars$temp)
```

What column contains temperature values in the dataframe `stars`. Identify where the new output column name goes, left or right of the =?

Restate the Goal. What are you trying to learn, check, or demonstrate?

```
#  Code
______ <- _____ %>%
  mutate(
    _____ = case_when(
      _____ < 4000 ~ "Cool",
      _____ <= 7500 ~ "Warm",
      _____ ~ "Hot"
    )
  )
```

```
#  Checks
table(stars_recode$temp_group)  # expect 3 levels
```

Comment on the output. Do you see the expected labels? How many "Cool", "Warm", and "Hot" stars are there? What did you learn or notice from the output? Did the code work as expected? If you needed help,

what kind of help did you use?

8.6 Task 5

Unit conversion with `mutate()`

Create `international_friendly_heights` overwriting the `height` column. *hint: inches × 2.54*

```
#  pre-check
heights <- dslabs::heights
head(heights)
```

Restate the Goal. What are you trying to learn, check, or demonstrate?

```
#  Code
international_friendly_heights <- heights %>%
  mutate(_____ = _____ * _____)
```

```
#  Checks
head(international_friendly_heights)
```

Comment on the output. Do you see a new column? Do the values look correct for a unit conversion from inches to centimeters? What did you learn or notice from the output? Did the code work as expected? If you needed help, what kind of help did you use?